

Неориентирани граф е крещена двойка $G=(V, E)$, където:

- V е непразно крайно множество, чиито елементи се наричат върхове
- $E \subseteq \{x \subseteq V: |x|=2\}$ е множество от ребра

Неориентирани мултиграф е крещена тройка $G=(V, E, f_G)$, където:

- V е непразно крайно множество от върхове и
- E е множество от ребра, $V \cap E = \emptyset$ *защо не искаме да е крайно?*
- $f_G: E \rightarrow \{x \subseteq V: |x|=2\}$

На всяко ребро f_G съпоставя множество от двата му края. Тази дефиниция не позволява прилик по време да се разшири и f_G да е $f_G: E \rightarrow \{x \subseteq V: 1 \leq |x| \leq 2\}$.

Ориентиран граф

Ориентиран граф е крещена двойка $G=(V, E)$, където:

- V е непразно крайно множество от върхове
- $E \subseteq V \times V$ и за всяко $v \in V$, $(v, v) \notin E$

Тази дефиниция дава ориентиран граф без прилики, и ако се наложат условията за всяко $v \in V$ $(v, v) \notin E$, тогава ползваме ориентиран граф с прилики

Ориентиран мултиграф е крещена тройка $G=(V, E, f_G)$, където:

- V е непразно крайно множество от върхове
- E е множество от ребра $V \cap E = \emptyset$
- $f_G: E \rightarrow V \times V$

Нека $G=(V, E, f_G)$ е неориентиран мултиграф. Път в G е алтернираща редица от върхове и ребра $p=(u_0, e_0, u_1, e_1, u_2, \dots, u_{k-1}, e_{k-1}, u_k)$, $k \geq 0$, $u_i \in V$, $e_{k_j} \in E$ и $f_G(e_{k_j}) = \{u_{i_j}, u_{i_{j+1}}\}$

Нека $G=(V, E, f_G)$ е ориентиран мултиграф. Път в G е алтернираща редица от върхове и ребра $p=(u_0, e_0, u_1, e_1, \dots, u_{k-1}, e_{k-1}, u_k)$, $k \geq 0$, $u_i \in V$, $e_{k_j} \in E$ и $f_G(e_{k_j}) = (u_{i_j}, u_{i_{j+1}})$

Ако всички върхове и ребра в път са уникални, то пътят е прост.

Цикъл е път в който $u_0 = u_k$. Прост цикъл е цикъл с поне 1 ребро и всичките му елементи са уникални с изключение на $u_0 = u_k$.

Свързаност в неориентиран граф

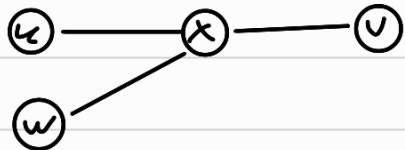
Два върха $u, v \in V$ са свързани ако в G съществува $u-v$ път. Графът G е свързан ако всеки два върха в него са свързани.

Свързаност е релация на екв.

- рефлексивност (всеки връх е свързан със себе си с път с дължина 0)
- симетричност (в неориентиран граф. ако имаме $u-v$, то $v-u$ съществува)
- транзитивност

Ако има прост $u-v$ път p и прост $v-w$ път q то нямаше значение ако ги обединим ще получим прост път

Например в този граф ако обединим p и q получаваме u, x, v, x, w не е прост



За това издириме от p най-близкия до u , който присъства в q и връщаме p' подвземайки p от u до x и q' подвземайки q от x до w и тях обединяваме.

В примера по-горе x е x и вместо u, x, v, x, w правим u, x, w .

Свързани компоненти са подграфите на G индуцирани от класовете на еквивалентност на релацията свързаност.

Два върха са силно свързани ако има път от u до v и от v до u . Силно свързаните компоненти са класовете на екв. на тази релация.

Слабо свързаните компоненти на ориентиран граф са свързаните компоненти на съответния неориентиран граф.

Дърво е свързан ациклически граф

Дърво може да се дефинира и индуктивно:

- Всеки граф с 1 връх и без ребра е дърво
- Ако $T=(V,E)$ е дърво и $u \in V$ и $w \notin V$, то $T'=(V \cup \{w\}, E \cup \{(u,w)\})$ също е дърво

Кореново дърво е дърво в което един връх $r \in V$ е избран за корен.

Кореново дърво може да се дефинира и индуктивно:

- Всеки граф само с 1 връх и без ребра е кореново дърво, н.е. $RT=(\{u\}, \emptyset)$
- Нека $T_1, T_2, \dots, T_k$ са коренови дървета с корени $r_1, \dots, r_k$ и дърветата T_i да не делят ният общи върхове. Нека r е нов връх, тогава графът в който r е свързан с $r_1, \dots, r_k$ е кореново дърво

• База ($|V|=1$)

Тривиалното кореново дърво $RT=(\{u\}, \emptyset)$ $|V|=1$ и $|E|=0$ и $1=0+1 \checkmark$

• ИХ Нека за всеки от $T_1, \dots, T_k$ е изпълнено $|V_{T_i}| = |E_{T_i}| + 1$

• ИС Нека r е нов връх, който свързваме с корените на $T_1, \dots, T_k$. Ще докажем T е кореново дърво. Нека делящи върховете на T с V_T и ребрата с E_T . Таки получаваме че $|V_T| = \underbrace{|V_{T_1}| + |V_{T_2}| + \dots + |V_{T_k}|}_{\text{Всички върхове от } T_1, \dots, T_k} + 1 = \sum_{i=1}^k |V_{T_i}| + 1$

$$|E_T| = \underbrace{|E_{T_1}| + |E_{T_2}| + \dots + |E_{T_k}|}_{\text{Всички ребра от } T_1, \dots, T_k} + \underbrace{k}_{\substack{\text{ребра } (r, r_i), i=1, \dots, k}} = \sum_{i=1}^k |E_{T_i}| + k$$

$$\text{Искам } |V_T| = |E_T| + 1 \Leftrightarrow |V_{T_1}| + |V_{T_2}| + \dots + |V_{T_k}| + 1 = |E_{T_1}| + |E_{T_2}| + \dots + |E_{T_k}| + k + 1$$

$$\text{От ИХ } \Rightarrow |V_{T_i}| = |E_{T_i}| + 1 \text{ заместваме в } |V_{T_1}| + |V_{T_2}| + \dots + |V_{T_k}| + 1 \text{ и получаваме } \underbrace{(|E_{T_1}| + 1) + (|E_{T_2}| + 1) + \dots + (|E_{T_k}| + 1)}_{\text{к пъти}} + 1 = |E_{T_1}| + |E_{T_2}| + \dots + |E_{T_k}| + k + 1, \text{ което е точно това което искахме}$$

Сеза трябва да докажем свързаност и ациклическост

(свързаност: От ИХ знаем, че $T_i, i=1, k$ са коренови дървета $\Rightarrow$ в частност са си свързани графи $\Rightarrow$ нека x и y са произволни върхове от два различни дървета — например $x \in T_i$ и $y \in T_j$, тогава или път p от x до r_i и или път q от r_j до y . Ние добавихме к ребра от r към всеки корен $\Rightarrow$ от r_i или път до r и от r или път до r_j и така за два произволни върха получихме път между тях $(x \rightarrow \dots \rightarrow r_i \rightarrow r \rightarrow r_j \rightarrow \dots \rightarrow y)$

Дзикалност: Да допуснем, че T има ^{прост} $\sqrt{k}$ цикла. Тогава той няма как да е изцяло да е в дърво $T_i, i=1, k$ защото това би било противоречие с ИХ. Така значи се използва линията през r , но единствените ребра инцидентни с r са (r, r_j) за $j=1, k$, а T_i нямат други върхове, т.е. простият $\sqrt{k}$ цикла би бил принуден да минае през r два пъти $\nexists$ с простотата на $\sqrt{k}$ цикла.

Покриващо дърво

Нека $G=(V, E)$ е свързан граф. Покриващо дърво на G е дърво $T=(V, E')$, където $E' \subseteq E$

```
distance[1..n] ← malloc(n) {+∞, ..., +∞}
```

```
color[1..n] ← malloc(n) {white, ..., white}
```

```
parent[1..n] ← malloc(n) {null, ..., null}
```

```
queue ← new Queue() // FIFO struct
```

```
color[1] = gray
```

```
queue.push(1)
```

```
while not queue.empty()
```

```
    node ← queue.pop()
```

```
    for child in adj(node)
```

```
        if color[child] = white
```

```
            color[child] = gray
```

```
            distance[child] = distance[node] + 1
```

```
            parent[child] = node
```

```
            queue.push(child)
```

endif

endfor

color[node] = black

endwhile

DFS(G)

color[1..n] $\leftarrow$ malloc(n) {white, ..., white}

parent[1..n] $\leftarrow$ malloc(n) {null, ..., null}

DFSvisiting(G, 1)

DFSVisiting(G, node)

color[node] = gray

foreach child in adj(node)

if color[child] = white

parent[child] = node

DFSvisiting(G, child)

endif

endfor

color[node] = black

И гвѣта алгоритма са $O(n+m)$

Ейлеров цикл в G е цикл (не непременно прост) който съдържа всяко ребро точно веднѣж

Ейлеров път е път (не непременно прост) който съдържа всяко ребро точно веднѣж

G е ейлеров ако в него има Ейлеров цикл

Теорема

(връзваният мултиграф G или Ойлеров уикъл $\Leftrightarrow$ всеки негов връх е четна степен

$\Rightarrow$ Неки $u \in V$ е произволен връх. Гледане на s като на кръгове наредби.

Таки всяка поява на u в s има точно два съседни елемента

Случай 1 (имаме прилики)

Ако няма прилики то всяко срещане на u в s има две ребра инцидентни с u . За различни срещания двойките ребра не се пресичат (s е ойлеров) и при t появи на u има 2 t ребра инцидентни с u , но това са всички инцидентни на u ребра (защото s е ойлеров) и така $d(u) = 2t$ - четно

Случай 2 (имаме прилики)

Ако има g прилики то случаите с прилики в уикъла са във вида $u, u, u, \dots$, а това увеличава степеня на u с 2. За останалите $t-g$ срещания без прилики важи случай 1 и така $d(u) = 2g + 2(t-g) = 2(t) = 2t$ - четно

$\Leftarrow$ Търваме от произволен връх $a \in V$ и на всяка стъпка минаваме по произв. неизползвано ребро инцидентно с текущия връх. Този процес може да продължава само в a защото всички върхове са четна степен и като влезем в тях винаги или откъде да излезем, н.е. можем да дойдем в a . Ако сме обходили всички ребра сме готови. В противен случай издираме връх u от който все още има неодомен инцидентни ребра (такива има защото G - свързан) и. Тонезис $d(u)$ изрочага едно четно (по усл.) след като се е завършил с u произвежда да е четно (четно - четно = четно) и издират 1 ребро и u за посетени от s (остават четни защото четно - четно = четно) уручане по усл.